GCP Enterprise / Principal Cloud Architect Playbook

Migration, Modernization & Greenfield — With AWS Comparisons

> **Author:** Pushparaj Naik **Scope:** Google Cloud Platform — End-to-End Lifecycle
> **Audience:** Enterprise Architects, Principal Cloud Architects, Multi-Cloud Strategists

---

Table of Contents

Part 1 — GCP vs AWS: Philosophical Differences

Before diving into GCP specifics, understand **why GCP feels different** from AWS. This is what interviewers test — can you articulate the fundamental architectural philosophy?

1.1 Core Philosophy Comparison

Aspect	AWS	GCP
Design Philosophy	"Here are 200+ services, pick what you need" — breadth-first	"Here are fewer, more opinionated services that integrate deeply" — depth-first
Network Model	Regional VPCs, explicit peering/Transit Gateway	**Global VPCs** (single VPC spans all regions) — fundamentally different
Identity Model	IAM users, roles, policies at account level	**Organization-wide IAM** with hierarchical resource manager
Compute Innovation	First-mover in cloud (EC2 2006), broadest compute options	Kubernetes (GKE invented K8s), serverless (Cloud Run), and data analytics (BigQuery)
Data/Analytics	Assembled from many services (S3 + Glue + Athena + Redshift)	**BigQuery** as a single unified analytics platform
AI/ML	Bedrock (managed LLMs), SageMaker (custom ML)	Vertex AI (unified ML), Gemini models, deepest AI research heritage
Pricing Model	On-demand, Reserved Instances, Savings Plans	On-demand, **Committed Use Discounts (CUDs)**, **Sustained Use Discounts (automatic)**
Multi-Cloud Story	"All-in on AWS" (limited multi-cloud tooling)	**Anthos** (run GKE anywhere: on-prem, AWS, Azure)
Open Source	Managed open-source services (OpenSearch, MSK)	**Founded on open source** (Kubernetes, TensorFlow, Istio, Knative)

1.2 When to Choose GCP Over AWS — The Architect's Decision

Scenario	Why GCP Wins	AWS Comparison
Big data / analytics	BigQuery is 5-10x simpler than the AWS equivalent stack	AWS needs S3 + Glue + Athena + Redshift + EMR
Kubernetes-first strategy	GKE is the gold standard (Google invented K8s)	EKS is good but GKE has auto-upgrade, auto-repair, release channels
AI/ML platform	Vertex AI + Gemini + TPUs = best integrated ML stack	SageMaker is powerful but more assembly required
Global networking	Global VPC, premium tier networking, private Google access	AWS VPCs are regional, need Transit Gateway for global
Multi-cloud strategy	Anthos runs GKE on AWS/Azure/on-prem	AWS has no equivalent multi-cloud runtime
Data warehouse / BI	BigQuery (serverless, pay-per-query, no cluster management)	Redshift needs cluster sizing, vacuuming, maintenance
Container-first with no K8s ops	Cloud Run (serverless containers, scale-to-zero)	App Runner is limited; ECS/Fargate doesn't scale to zero
Cost-sensitive steady workloads	Sustained Use Discounts apply automatically (no commitment)	AWS requires explicit RI/SP purchasing

1.3 When AWS Still Wins Over GCP

Scenario	Why AWS Wins	GCP Gap
Broadest service catalog	200+ services for every niche use case	GCP has ~100 core services
Enterprise maturity	Longer enterprise track record, more compliance certifications	GCP catching up but AWS has deeper govt/regulated sector footprint
IoT / Edge	IoT Core + Greengrass + SiteWise ecosystem	GCP deprecated IoT Core in Aug 2023

Scenario	Why AWS Wins	GCP Gap
Marketplace / ISV ecosystem	Largest cloud marketplace	GCP Marketplace is smaller
Hybrid with VMware	VMware Cloud on AWS is mature	Google Cloud VMware Engine exists but less adopted
Serverless database	DynamoDB is unmatched for key-value at scale	Firestore is good but less enterprise-adopted

Part 2 — GCP Service Mapping (vs AWS)

2.1 Compute

Category	AWS	GCP	Key Difference
Virtual Machines	EC2	**Compute Engine**	GCP has live migration (no downtime for host maintenance)
Managed Kubernetes	EKS	**GKE (Google Kubernetes Engine)**	GKE has auto-upgrade, auto-repair, release channels, Autopilot mode
Serverless Containers	ECS Fargate / App Runner	**Cloud Run**	Cloud Run scales to zero, supports any container, built on Knative
Serverless Functions	Lambda	**Cloud Functions (2nd gen)**	Cloud Functions 2nd gen is built on Cloud Run
Batch Processing	AWS Batch	**Batch**	Similar capabilities
VM Images	AMI	**Machine Images / Custom Images**	Similar
Spot/Preemptible	Spot Instances	**Spot VMs (Preemptible VMs)**	GCP Spot VMs have no minimum runtime (AWS Spot has 2-min warning)

2.2 Containers & Kubernetes

Category	AWS	GCP	Key Difference
Managed K8s	EKS	**GKE Standard**	GKE is the most mature managed K8s (Google invented it)
Serverless K8s	EKS + Fargate	**GKE Autopilot**	Autopilot = fully managed, per-pod pricing, no node management
Container Registry	ECR	**Artifact Registry**	Artifact Registry supports Docker, Maven, npm, Python, Go
Service Mesh	App Mesh / Istio on EKS	**Anthos Service Mesh (managed Istio)**	Fully managed Istio with mTLS, traffic management
Multi-Cloud K8s	No equivalent	**Anthos**	Run GKE on AWS, Azure, bare metal, edge
Serverless Containers	App Runner / Fargate	**Cloud Run**	Cloud Run = best serverless container experience

2.3 Database & Storage

Category	AWS	GCP	Key Difference
Object Storage	S3	**Cloud Storage (GCS)**	Single global namespace, similar tiering (Standard/Nearline/Coldline/Archive)
Block Storage	EBS	**Persistent Disk (PD)**	GCP PD can be attached to multiple VMs (read-only multi-attach)
Managed PostgreSQL/MySQL	RDS / Aurora	**Cloud SQL**	Cloud SQL supports PostgreSQL, MySQL, SQL Server. No Aurora equivalent
Distributed SQL	Aurora Global Database	**Cloud Spanner**	Spanner = globally distributed, strongly consistent, horizontally scaled SQL (unique!)
NoSQL (Document)	DynamoDB	**Firestore**	Firestore has real-time sync, offline support, better for mobile

Category	AWS	GCP	Key Difference
NoSQL (Wide Column)	DynamoDB / Keyspaces	**Bigtable**	Bigtable = petabyte-scale, single-digit ms latency, used by Google Search/Maps
In-Memory Cache	ElastiCache (Redis/Memcached)	**Memorystore**	Memorystore supports Redis and Memcached
Data Warehouse	Redshift	**BigQuery**	BigQuery = serverless, pay-per-query, no cluster management (game-changer)

2.4 Networking

Category	AWS	GCP	Key Difference
VPC	Regional VPC	**Global VPC**	GCP VPCs span all regions automatically (huge difference)
Load Balancer	ALB / NLB / GLB	**Cloud Load Balancing (Global)**	GCP has a single global L7 load balancer with anycast IPs
CDN	CloudFront	**Cloud CDN**	Integrated with global LB, Media CDN for streaming
DNS	Route 53	**Cloud DNS**	Similar capabilities
VPN	Site-to-Site VPN	**Cloud VPN (HA VPN)**	99.99% SLA with HA VPN
Dedicated Connectivity	Direct Connect	**Cloud Interconnect (Dedicated/Partner)**	Similar — 10G/100G dedicated links
Transit/Hub	Transit Gateway	**NCC (Network Connectivity Center)**	NCC is hub-and-spoke for hybrid/multi-cloud
Private Service Access	VPC Endpoints (PrivateLink)	**Private Google Access / Private Service Connect**	Similar pattern, different implementation
Firewall	Security Groups + NACLs	**VPC Firewall Rules / Firewall Policies**	GCP firewall rules use tags and service accounts (more flexible)

Category	AWS	GCP	Key Difference
Network Security	AWS Network Firewall	**Cloud NGFW (Next-Gen Firewall)**	Palo Alto-powered L7 inspection

2.5 Security & Identity

Category	AWS	GCP	Key Difference
Identity	IAM Users/Roles/Policies	**Cloud IAM (Organization-level)**	GCP IAM is hierarchical: Org → Folder → Project → Resource
SSO/Federation	IAM Identity Center	**Cloud Identity / Workspace**	Integrates with Google Workspace, external IdPs
Secrets	Secrets Manager	**Secret Manager**	Similar capabilities, integrated with Cloud Run/GKE
Key Management	KMS	**Cloud KMS**	Both support CMKs, HSM-backed keys
Certificate Management	ACM	**Certificate Manager**	Auto-provisioned, managed SSL certificates
Threat Detection	GuardDuty	**Security Command Center (SCC)**	SCC Premium includes threat detection, vulnerability scanning, compliance
Security Posture	Security Hub	**SCC Premium**	Unified security dashboard, similar to Security Hub
DDoS Protection	AWS Shield	**Cloud Armor**	Cloud Armor = WAF + DDoS combined (vs AWS Shield + WAF separate)
Organization Policies	SCPs	**Organization Policy Constraints**	Similar preventive controls at org/folder/project level

Category	AWS	GCP	Key Difference
Audit Logging	CloudTrail	**Cloud Audit Logs**	Always-on admin activity logs (no configuration needed!)

2.6 DevOps & CI/CD

Category	AWS	GCP	Key Difference
Source Repository	CodeCommit (deprecated)	**Cloud Source Repositories**	Both being replaced by GitHub/GitLab
CI/CD Pipeline	CodePipeline + CodeBuild	**Cloud Build**	Cloud Build is simpler, YAML-based, integrates with GKE/Cloud Run
Artifact Storage	ECR + CodeArtifact	**Artifact Registry**	Unified registry for Docker, npm, Maven, Python, Go
GitOps	ArgoCD on EKS	**Config Sync (Anthos Config Management)**	Config Sync = native GitOps for GKE
Infrastructure as Code	CloudFormation / Terraform	**Deployment Manager / Terraform**	Terraform is the de facto standard for GCP (Deployment Manager is legacy)
Monitoring	CloudWatch	**Cloud Monitoring (formerly Stackdriver)**	Cloud Monitoring has built-in SLO monitoring, uptime checks
Logging	CloudWatch Logs	**Cloud Logging**	Cloud Logging has advanced query language, log-based metrics
Tracing	X-Ray	**Cloud Trace**	Cloud Trace supports OpenTelemetry natively
Error Reporting	No direct equivalent	**Error Reporting**	Automatic error grouping and notification

2.7 AI/ML & Data Analytics

Category	AWS	GCP	Key Difference
ML Platform	SageMaker	**Vertex AI**	Vertex AI = unified platform (training, serving, pipelines, feature store, model registry)
Foundation Models	Bedrock (Claude, Titan, Llama)	**Vertex AI + Gemini**	Gemini is Google's frontier model, Vertex AI also hosts open models
Data Warehouse	Redshift	**BigQuery**	BigQuery ML = run ML directly in SQL. No ETL to separate ML platform
Stream Processing	Kinesis	**Dataflow (Apache Beam)**	Dataflow = unified batch + stream processing (same code)
ETL / Data Integration	Glue	**Dataflow / Dataproc / Cloud Data Fusion**	Cloud Data Fusion = visual ETL (CDAP-based)
Data Catalog	Glue Data Catalog	**Dataplex / Data Catalog**	Dataplex = data mesh governance across lakes and warehouses
Pub/Sub (Messaging)	SNS + SQS + EventBridge	**Pub/Sub**	Pub/Sub = unified messaging (replaces SNS+SQS+EventBridge)
Workflow Orchestration	Step Functions	**Workflows / Cloud Composer (Airflow)**	Cloud Composer = managed Apache Airflow
Vector Search	OpenSearch Serverless / pgvector	**Vertex AI Vector Search / AlloyDB AI**	AlloyDB has built-in pgvector with Google-optimized indexes

Part 3 — GCP Organization & Landing Zone

3.1 GCP Resource Hierarchy — Fundamentally Different from AWS

```
GCP RESOURCE HIERARCHY (vs AWS):

┌──────────────────────────────────────────────────────────────────────┐
│  GCP                                    AWS Equivalent               │
│                                                                      │
│  Organization (domain-level)            ≈ AWS Organizations          │
│  ├── Folders (nested, unlimited depth)  ≈ OUs (less flexible)        │
│  │   ├── Folder: Production             ≈ Production OU              │
│  │   │   ├── Project: prod-app1         ≈ AWS Account: prod-app1     │
│  │   │   ├── Project: prod-app2         ≈ AWS Account: prod-app2     │
│  │   │   └── Project: prod-shared-vpc   ≈ AWS Account: network-hub    │
│  │   ├── Folder: Non-Production                                      │
│  │   │   ├── Project: dev-app1                                       │
│  │   │   └── Project: staging-app1                                   │
│  │   ├── Folder: Security                                            │
│  │   │   ├── Project: security-logging                               │
│  │   │   └── Project: security-monitoring                            │
│  │   └── Folder: Shared Services                                     │
│  │       ├── Project: shared-networking                              │
│  │       ├── Project: shared-cicd                                    │
│  │       └── Project: shared-dns                                     │
│  └── Organization Policies             ≈ SCPs                        │
│      └── Applied at Org/Folder/Project  ≈ Applied at Org/OU/Account  │
└──────────────────────────────────────────────────────────────────────┘

KEY DIFFERENCES:
• GCP "Project" = AWS "Account" (the blast radius boundary)
• GCP "Folders" can nest deeply (AWS OUs are more limited)
• GCP IAM is HIERARCHICAL — permissions cascade down from Org → Folder → Project
• GCP has NO per-project root user (unlike AWS per-account root user problem)
```

3.2 GCP Landing Zone Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                  GCP LANDING ZONE (Fabric FAST / CFT)                │
│                                                                      │
│  ORGANIZATION: company.com                                           │
│  ├── Org-Level Policies:                                             │
│  │   ├── constraints/compute.disableSerialPortAccess = TRUE          │
│  │   ├── constraints/compute.requireShieldedVm = TRUE                │
│  │   ├── constraints/iam.disableServiceAccountKeyCreation = TRUE     │
│  │   ├── constraints/storage.uniformBucketLevelAccess = TRUE         │
│  │   ├── constraints/gcp.resourceLocations = [us-central1, europe-west1]│
│  │   └── constraints/compute.vmExternalIpAccess = DENY_ALL           │
│  │                                                                   │
│  ├── Folder: Bootstrap                                               │
│  │   └── Project: bootstrap-seed (Terraform state, CI/CD service accts) │
│  │                                                                   │
│  ├── Folder: Security                                                │
│  │   ├── Project: security-logging (centralized Cloud Audit Logs sink)  │
│  │   ├── Project: security-monitoring (SCC, Chronicle SIEM)          │
│  │   └── Project: security-kms (shared KMS keys)                     │
│  │                                                                   │
│  ├── Folder: Networking                                              │
│  │   ├── Project: shared-vpc-prod (Host VPC for production)          │
```

```
│   │   ├── Project: shared-vpc-nonprod (Host VPC for dev/staging)          │
│   │   └── Project: network-connectivity (Cloud Interconnect, VPN)         │
│   │                                                                       │
│   ├── Folder: Production                                                  │
│   │   ├── Project: prod-app1 (Service Project attached to Shared VPC)     │
│   │   ├── Project: prod-app2                                              │
│   │   └── Project: prod-data                                              │
│   │                                                                       │
│   ├── Folder: Non-Production                                              │
│   │   ├── Project: dev-app1                                               │
│   │   └── Project: staging-app1                                           │
│   │                                                                       │
│   └── Folder: Sandbox                                                     │
│       ├── Project: sandbox-engineer1                                      │
│       └── Project: sandbox-engineer2                                      │
└───────────────────────────────────────────────────────────────────────────┘
```

3.3 Shared VPC — GCP's Network Hub Model

```
SHARED VPC (vs AWS Transit Gateway):

AWS Model:                                    GCP Model:

┌──────────┐                                  ┌──────────────────────────────┐
│ Account A │──┐                              │ Host Project (Shared VPC)    │
│ (own VPC) │  │   ┌─────────┐                │                              │
└──────────┘  ├──▶│ Transit │                │ VPC: 10.0.0.0/16             │
┌──────────┐  │   │ Gateway │                │ ├── Subnet: prod (10.0.1.0/24)
│ Account B │──┤   └─────────┘                │ ├── Subnet: dev  (10.0.2.0/24)
│ (own VPC) │  │                              │ └── Subnet: data (10.0.3.0/24)
└──────────┘  │                              └──────────────┬───────────────┘
┌──────────┐  │                                             │ Shared to:
│ Account C │──┘                                ┌──────────┼──────────┐
│ (own VPC) │                                   ▼          ▼          ▼
└──────────┘                              Service Proj A  Service Proj B  Service Proj C
                                          (uses prod      (uses dev       (uses data
                                           subnet)         subnet)         subnet)

KEY DIFFERENCE:
• AWS: Each account has its own VPC → connect via Transit Gateway
• GCP: Host Project owns the VPC → Service Projects use subnets from it
• GCP Shared VPC is SIMPLER (no routing, no peering, no TGW)
• But AWS TGW is MORE FLEXIBLE (supports more complex topologies)
```

3.4 Landing Zone Automation Tools

Tool	GCP	AWS Equivalent
Landing Zone Blueprint	**Cloud Foundation Toolkit (CFT)** or **Fabric FAST**	Control Tower

Tool	GCP	AWS Equivalent
Policy-as-Code	Organization Policy Constraints + Terraform	SCPs + CloudFormation
Guardrails	Organization Policies (preventive) + SCC (detective)	SCPs + Config Rules
Account/Project Factory	Fabric FAST Project Factory	Control Tower Account Factory
IaC	**Terraform** (de facto standard for GCP)	Terraform / CloudFormation

Part 4 — GCP Migration: Assess → Plan → Deploy → Optimize

4.1 GCP Migration Framework (vs AWS MAP)

```
GCP MIGRATION PHASES:

AWS MAP:     Assess  →  Mobilize  →  Migrate & Modernize
GCP:         Assess  →  Plan      →  Deploy   →  Optimize

Both follow the same logical flow, just different naming:

Phase 1: ASSESS
├── Google Cloud Migration Assessment Tool (mFIT)
├── StratoZone (portfolio discovery)
├── Cloud Adoption Framework for GCP
└── TCO Calculator

Phase 2: PLAN
├── Foundation deployment (Landing Zone)
├── Network connectivity (Cloud Interconnect / VPN)
├── Migration wave planning
└── Team training (Google Cloud Skills Boost)

Phase 3: DEPLOY
├── Migrate to Virtual Machines (VM migration)
├── Database Migration Service (DMS)
├── BigQuery Data Transfer Service
├── Transfer Appliance (offline data transfer)
└── Storage Transfer Service

Phase 4: OPTIMIZE
├── Right-sizing recommendations (Recommender)
├── Committed Use Discounts
```

```
├── Active Assist recommendations
└── Cost management (Cloud Billing)
```

4.2 GCP Migration Tools (vs AWS)

Migration Task	GCP Tool	AWS Equivalent
Portfolio Discovery	StratoZone / mFIT	Application Discovery Service / Migration Evaluator
VM Migration	**Migrate to Virtual Machines (M2VM)**	Application Migration Service (MGN)
Container Migration	**Migrate to Containers (M2C)**	No direct equivalent (manual containerization)
Database Migration	**Database Migration Service (DMS)**	AWS DMS
Schema Conversion	Database Migration Service (built-in)	Schema Conversion Tool (SCT) — separate tool
Large Data Transfer	**Transfer Appliance** (100TB/300TB)	Snowball / Snowmobile
Online Data Transfer	**Storage Transfer Service**	DataSync
BigQuery Loading	**BigQuery Data Transfer Service**	No equivalent (manual ETL to Redshift)
Tracking Dashboard	**Migration Center**	Migration Hub

4.3 The 6 R's on GCP — Same Framework, Different Tools

Strategy	GCP Implementation	AWS Comparison
Rehost (Lift & Shift)	Migrate to Virtual Machines (M2VM) → Compute Engine	AWS MGN → EC2
Replatform	Move MySQL → Cloud SQL, Deploy on GKE instead of VMs	Move MySQL → RDS, Deploy on EKS

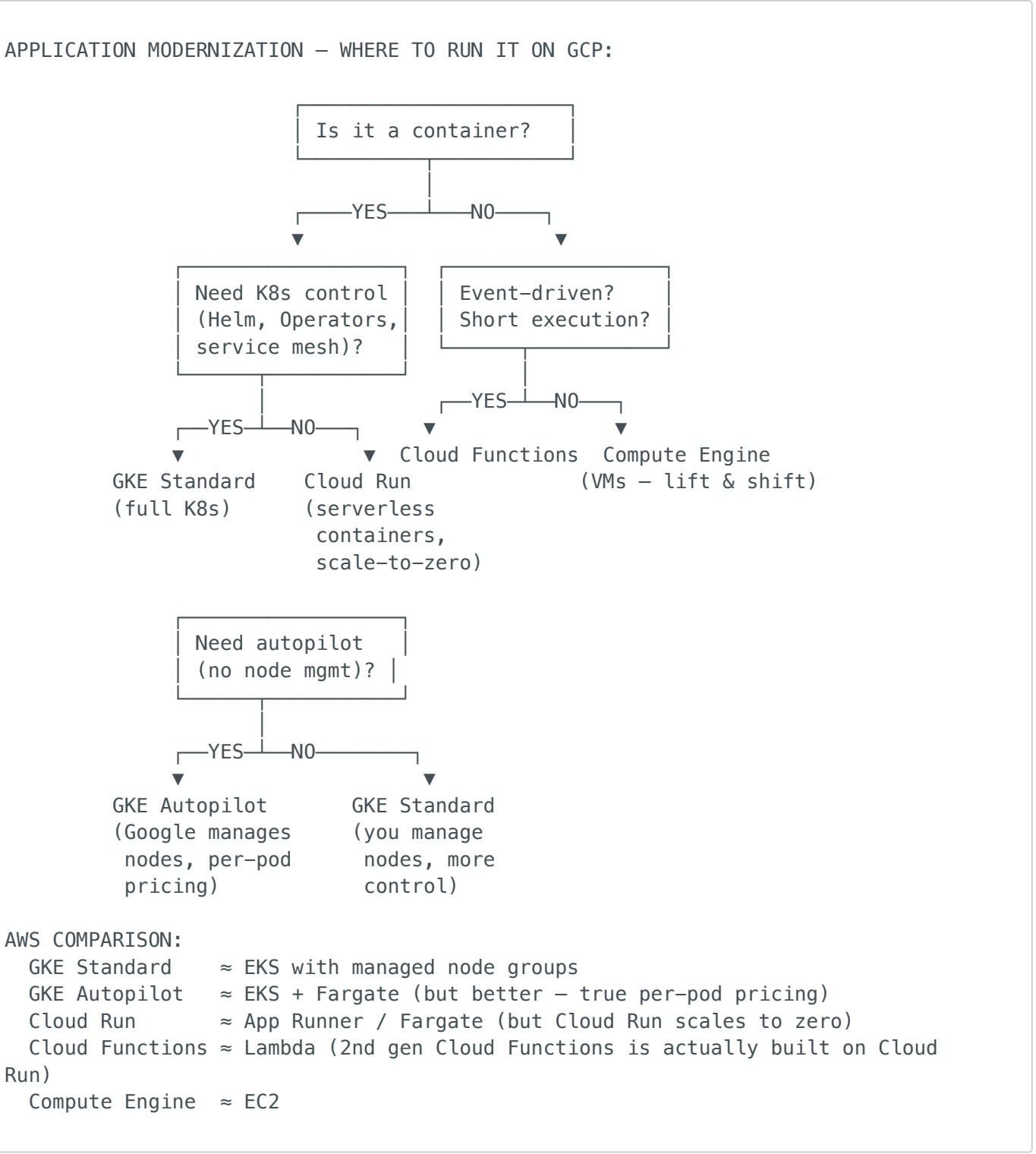
Strategy	GCP Implementation	AWS Comparison
Refactor	Rewrite for Cloud Run, GKE, Firestore, Pub/Sub	Rewrite for ECS/Lambda, DynamoDB, SQS
Repurchase	Google Workspace, Looker, Chronicle SIEM	O365, QuickSight, GuardDuty
Retain	Keep on-prem, connect via Cloud Interconnect	Keep on-prem, connect via Direct Connect
Retire	Decommission — same on both platforms	Decommission

4.4 Database Migration — GCP Specifics

```
DATABASE MIGRATION DECISION ON GCP:

Source Database         → GCP Target                    → AWS Equivalent Target
──────────────────────────────────────────────────────────────────────────────
Oracle                  → Cloud SQL PostgreSQL          → Aurora PostgreSQL
                        → AlloyDB (Oracle-compatible)   → (No direct equivalent)
                        → Cloud Spanner (global scale)  → Aurora Global Database

SQL Server              → Cloud SQL SQL Server          → RDS SQL Server
                        → Cloud SQL PostgreSQL          → Aurora PostgreSQL

MySQL                   → Cloud SQL MySQL               → RDS MySQL / Aurora MySQL

PostgreSQL              → Cloud SQL PostgreSQL          → RDS PostgreSQL / Aurora
                        → AlloyDB (high-performance)    → Aurora

MongoDB                 → MongoDB Atlas on GCP          → DocumentDB / Atlas on AWS
                        → Firestore                     → DynamoDB

Mainframe DB            → Cloud Spanner                 → Aurora / DynamoDB

GCP UNIQUE ADVANTAGE:
• AlloyDB = PostgreSQL-compatible with 4x faster transactions, 100x faster
analytics
  (No AWS equivalent — Aurora is the closest but AlloyDB is specifically designed
  for complex queries that need both OLTP and OLAP)
• Cloud Spanner = globally distributed SQL with strong consistency
  (No AWS equivalent — Aurora Global is eventually consistent cross-region)
```

Part 5 — GCP Modernization & Cloud-Native Patterns

5.1 Compute Decision Tree on GCP



5.2 GKE Deep Dive — Why It's the Gold Standard

Feature	GKE	EKS	Advantage
Release Channels	Rapid, Regular, Stable (auto-managed)	Manual version selection	GKE auto-upgrades safely
Auto-Repair	Detects and repairs unhealthy nodes	Manual (or custom scripts)	GKE handles node health

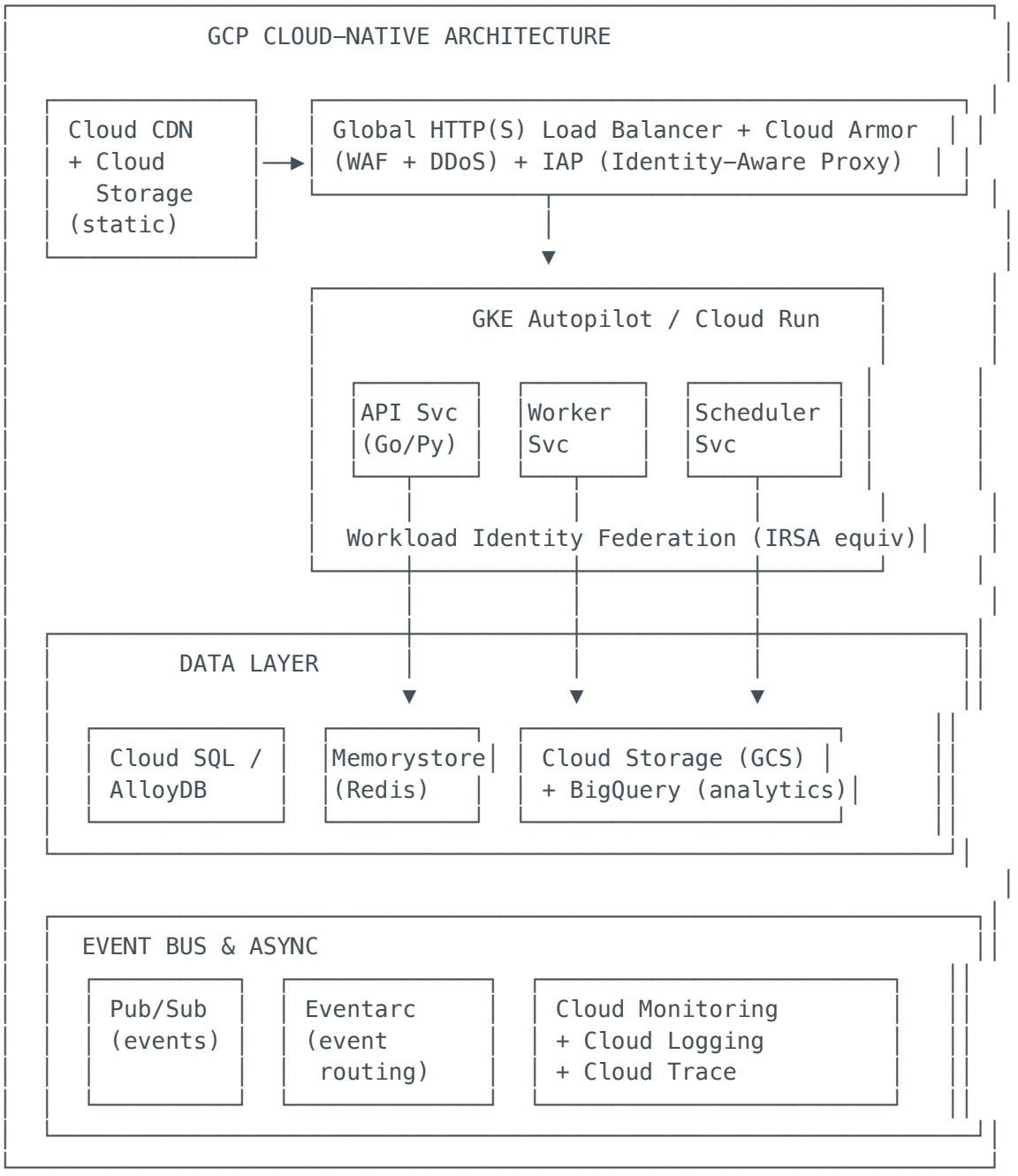
Feature	GKE	EKS	Advantage
	automatically		
Autopilot Mode	True serverless K8s — per-pod pricing, no node ops	Fargate profiles (more limited)	Autopilot = simpler
Workload Identity	Pod → Google Cloud IAM binding	IRSA (similar, more setup)	Same concept, GKE is simpler
Config Sync	Native GitOps built-in	ArgoCD (third-party)	Built-in vs. add-on
Binary Authorization	Enforce signed container images	No native equivalent	Supply chain security
GKE Sandbox (gVisor)	Kernel-level container isolation	No native equivalent	Extra security layer
Multi-Cluster Ingress	Global load balancing across GKE clusters	Manual with Route53	Native multi-cluster
Cost	GKE Standard: $0.10/hr/cluster. Autopilot: per-pod	EKS: $0.10/hr/cluster + Fargate pricing	Similar control plane cost

5.3 Cloud Run — GCP's Secret Weapon

```
CLOUD RUN vs AWS EQUIVALENTS:

Cloud Run Feature              AWS Closest         Why Cloud Run Wins
──────────────────────────────────────────────────────────────────────
Scale to zero                  App Runner          App Runner limited; Fargate
can't
Any container image            App Runner          App Runner has size limits
Concurrency control            Lambda              Lambda = 1 request/instance
Request timeout up to 60 min   Lambda (15 min max) 4x longer execution
Min instances (warm)           Lambda Provisioned  Same concept
Traffic splitting              No built-in         Canary deployments built-in
Custom domains + SSL           Manual ACM + ALB    Auto-provisioned
Connect to VPC                 VPC Lambda/Fargate  Private Google Access
gRPC support                   Limited             Full gRPC support
WebSocket support              API GW WebSocket    Native WebSocket
Jobs (batch)                   AWS Batch / Lambda  Cloud Run Jobs for batch

WHEN TO USE CLOUD RUN:
├── Web APIs and microservices
├── Event-driven processing (Pub/Sub, Eventarc triggers)
├── Scheduled jobs (Cloud Scheduler → Cloud Run Jobs)
├── Webhook handlers
```

```
├── Internal microservices behind a service mesh
└── Any containerized workload that doesn't need K8s complexity
```

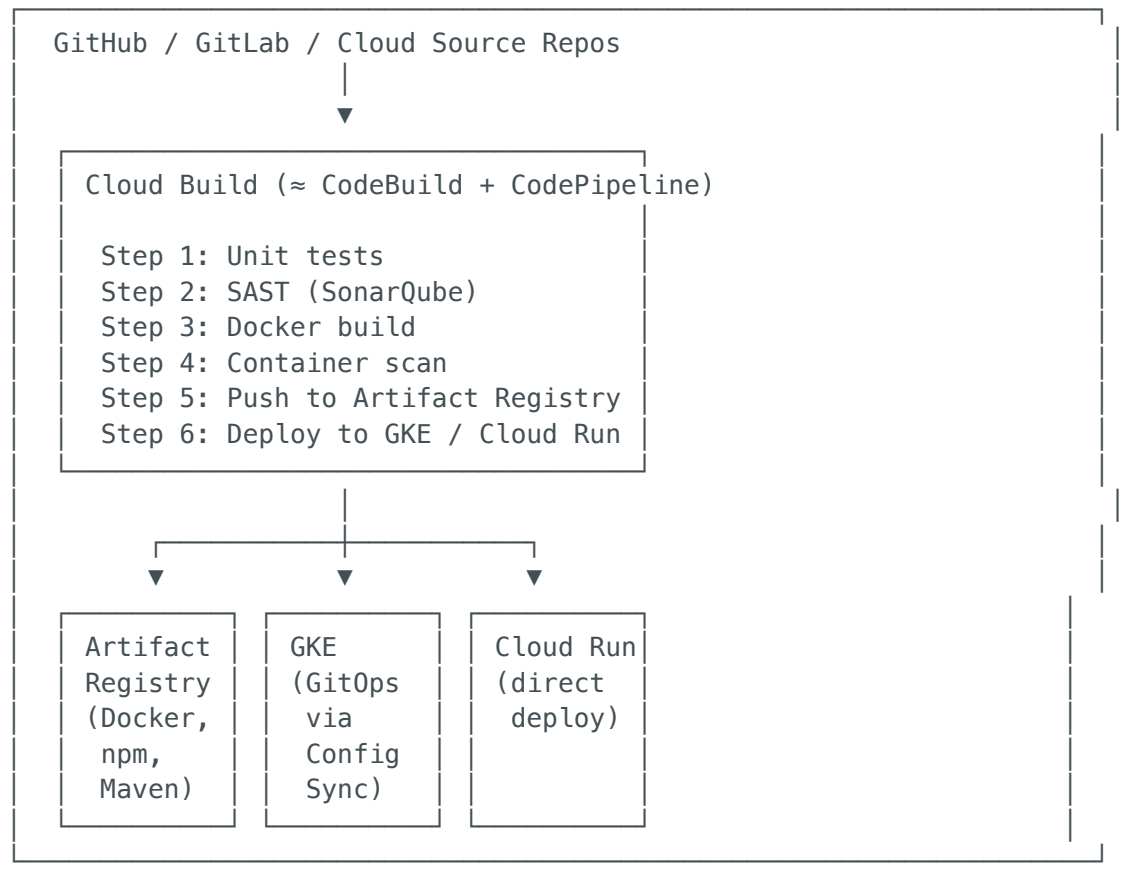
5.4 Reference Architecture — Cloud-Native on GCP




```
AWS EQUIVALENT:
  Global LB + Cloud Armor    ≈  CloudFront + WAF + Shield
  GKE Autopilot / Cloud Run  ≈  EKS Fargate / App Runner
  Cloud SQL / AlloyDB        ≈  Aurora PostgreSQL
  Memorystore                ≈  ElastiCache
  Pub/Sub + Eventarc         ≈  SNS + SQS + EventBridge
  Cloud Monitoring           ≈  CloudWatch
```

Part 6 — GCP DevOps & CI/CD Architecture

6.1 GCP-Native CI/CD Pipeline



6.2 GCP Monitoring & Observability Stack

Capability	GCP Service	AWS Equivalent	Key Difference
Metrics	Cloud Monitoring	CloudWatch Metrics	GCP has built-in SLO monitoring
Logging	Cloud Logging	CloudWatch Logs	GCP Logging = always-on audit logs, log-based metrics

Capability	GCP Service	AWS Equivalent	Key Difference
Tracing	Cloud Trace	X-Ray	Native OpenTelemetry support
Profiling	Cloud Profiler	No equivalent	Continuous production profiling (CPU, memory)
Error Tracking	Error Reporting	No equivalent	Auto-groups errors, sends notifications
Uptime Checks	Cloud Monitoring	Route 53 Health Checks	Built into monitoring, not DNS
Dashboards	Cloud Monitoring Dashboards	CloudWatch Dashboards	Similar
Alerting	Cloud Monitoring Alerts	CloudWatch Alarms	GCP supports SLO-based alerting
APM	Cloud Trace + Profiler	X-Ray + No profiler	GCP has more complete native APM

Part 7 — GCP Security, Compliance & Governance

7.1 Security Architecture Comparison

```
GCP SECURITY MODEL vs AWS:

GCP                                          AWS
_____________________________________________

_____________________________________________
Organization Policies (preventive)           SCPs (preventive)
├── Restrict resource locations              ├── Deny unapproved regions
├── Disable service account key creation     ├── Require IMDSv2
├── Require Shielded VMs                     ├── Require encryption
└── Uniform bucket access only               └── Block public S3

VPC Service Controls (data exfil prev)       VPC Endpoints (different concept)
├── Create a "perimeter" around projects     ├── Private connectivity to services
├── Prevent data from leaving the perimeter  ├── No equivalent data perimeter
└── Block API calls from outside             └── IAM policies for access control

Security Command Center (detective)          Security Hub + GuardDuty (detective)
├── Vulnerability scanning                   ├── Config Rules compliance
├── Threat detection (Event Threat Detection)├── GuardDuty threat detection
├── Compliance monitoring (CIS, PCI)         ├── Security Hub compliance
├── Web Security Scanner                     ├── Inspector
```

```
└── Container Threat Detection              └── No container-specific detection

Binary Authorization (supply chain)         No native equivalent
├── Require signed container images          ├── (Need Sigstore/Cosign manually)
├── Attestation-based deployment control     └──
└── Enforce in GKE/Cloud Run

GCP UNIQUE SECURITY ADVANTAGES:
1. VPC Service Controls — No AWS equivalent. Prevents data exfiltration at API
level
2. Binary Authorization — Native supply chain security for containers
3. Confidential Computing — Confidential VMs (encrypt data in-use, not just at-
rest)
4. BeyondCorp Enterprise — Zero-trust access proxy (IAP) built into GCP
5. Chronicle SIEM — Google-scale security analytics (not just GCP, all sources)
```

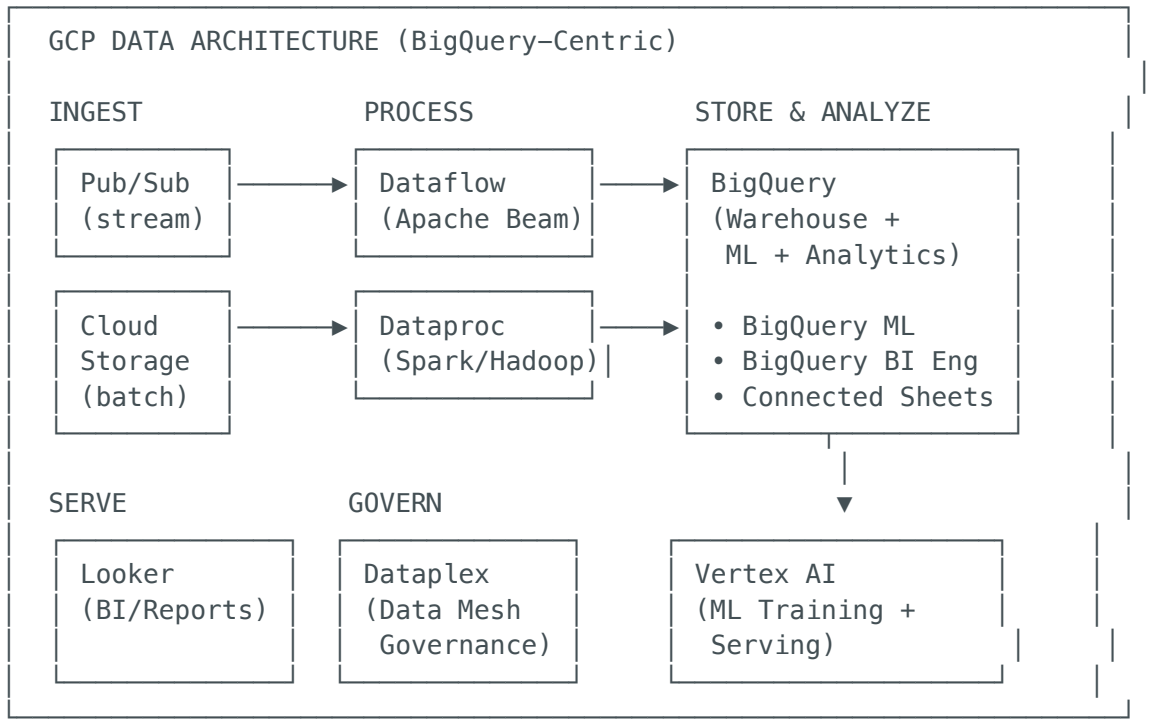
7.2 Identity & Access Management — GCP vs AWS

```
GCP IAM MODEL:

┌──────────────────────────────────────────────────────────────────┐
│  Organization: company.com                                       │
│  ├── IAM Policy: SecurityAdmin@company.com → roles/orgAdmin      │
│  │                                                               │
│  ├── Folder: Production                                          │
│  │   ├── IAM Policy: prod-team@company.com → roles/editor        │
│  │   │                                                           │
│  │   └── Project: prod-app1                                      │
│  │       ├── IAM Policy: app1-sa@prod-app1 → roles/cloudsql      │
│  │       └── Resources inherit ALL policies from above           │
│  │                                                               │
│  └── Folder: Sandbox                                             │
│      └── IAM Policy: devs@company.com → roles/editor             │
│          (Policies DON'T leak to Production folder)              │
└──────────────────────────────────────────────────────────────────┘

KEY DIFFERENCES FROM AWS:
• GCP IAM policies are ADDITIVE and INHERITED down the hierarchy
• GCP has NO IAM users — only Google accounts, groups, service accounts
• GCP service accounts ARE resources (can be impersonated, unlike AWS roles)
• GCP has Workload Identity Federation (like AWS OIDC but more mature)
• GCP IAM Conditions = attribute-based access (like AWS IAM Conditions)
```

Part 8 — GCP Data & AI/ML Platform Architecture

8.1 GCP Data Platform — The Crown Jewel

```
GCP DATA PLATFORM (vs AWS):
```




```
AWS EQUIVALENT STACK (requires 6+ services to match BigQuery alone):
  Pub/Sub        ≈ Kinesis Data Streams + SQS + SNS + EventBridge
  Dataflow       ≈ Kinesis Data Analytics / EMR (Spark)
  BigQuery       ≈ Redshift + Athena + Glue Data Catalog (COMBINED)
  BigQuery ML    ≈ SageMaker (separate service, separate skill set)
  Dataproc       ≈ EMR
  Dataplex       ≈ Lake Formation + Glue Data Catalog
  Looker         ≈ QuickSight (Looker is more powerful)
  Vertex AI      ≈ SageMaker

WHY GCP WINS FOR DATA:
• BigQuery is ONE service that replaces Redshift + Athena + Glue Catalog
• BigQuery ML lets analysts train ML models using SQL (no Python needed)
• Dataflow unifies batch + stream processing (same Apache Beam code)
• Pub/Sub replaces SNS + SQS + EventBridge (one service, simpler)
```

8.2 AI/ML on GCP — Vertex AI vs SageMaker

Capability	Vertex AI (GCP)	SageMaker (AWS)
Foundation Models	Gemini (1.5 Pro/Flash), PaLM, Imagen, Codey	Bedrock (Claude, Titan, Llama, Mistral)
Custom Training	Custom training jobs, hyperparameter tuning	Training jobs, HPO

Capability	Vertex AI (GCP)	SageMaker (AWS)
AutoML	AutoML (Vision, NLP, Tables, Video)	Autopilot
Feature Store	Vertex Feature Store	SageMaker Feature Store
Model Registry	Vertex Model Registry	SageMaker Model Registry
Pipelines	Vertex Pipelines (Kubeflow-based)	SageMaker Pipelines
Serving	Vertex Endpoints (auto-scaling, A/B testing)	SageMaker Endpoints
Vector Search	Vertex AI Vector Search (ScaNN-based)	OpenSearch / pgvector
RAG	Vertex AI Search + Grounding	Bedrock Knowledge Bases
Responsible AI	Vertex AI Responsible AI Toolkit	Bedrock Guardrails
TPU Access	**TPU v5e/v5p** (Google's custom ML hardware)	Trainium / Inferentia
Notebooks	Vertex AI Workbench (managed JupyterLab)	SageMaker Studio

Part 9 — GCP Networking Deep Dive

9.1 Global VPC — The Fundamental Difference

```
GCP GLOBAL VPC (vs AWS Regional VPC):

AWS:
  Region us-east-1:  VPC-A (10.0.0.0/16)
  Region eu-west-1:  VPC-B (10.1.0.0/16)  ← Separate VPC! Need peering/TGW
  Region ap-south-1: VPC-C (10.2.0.0/16)  ← Separate VPC! Need peering/TGW

GCP:
  Global VPC: my-vpc (spans ALL regions automatically)
  ├── Subnet: us-central1    (10.0.1.0/24)
  ├── Subnet: europe-west1   (10.0.2.0/24)  ← Same VPC! No peering needed
  ├── Subnet: asia-south1    (10.0.3.0/24)  ← Same VPC! Internal routing works
  └── Routes automatically configured

WHY THIS MATTERS:
• VM in us-central1 can ping VM in europe-west1 via INTERNAL IP (no peering!)
• GKE pods in different regions can communicate natively
• Firewall rules apply globally (one rule protects all regions)
```

```
• No Transit Gateway equivalent needed for multi-region
• Simpler, fewer moving parts, less to misconfigure
```

9.2 Load Balancing — GCP's Advantage

```
GCP GLOBAL LOAD BALANCING:

  Client (anywhere in the world)
         │
         ▼
  ┌──────────────────────────────────────────────┐
  │ Google Front End (GFE)                       │
  │ Single Anycast IP: 34.120.x.x                │
  │ Terminates SSL at Google's edge (175+ PoPs)  │
  │                                              │
  │ Routes to nearest healthy backend:           │
  │   ├── us-central1: GKE / Cloud Run           │
  │   ├── europe-west1: GKE / Cloud Run          │
  │   └── asia-east1: GKE / Cloud Run            │
  └──────────────────────────────────────────────┘

AWS EQUIVALENT REQUIRES:
  CloudFront → ALB (us-east-1) → EKS
               ALB (eu-west-1) → EKS  (separate ALBs per region)
               ALB (ap-south-1) → EKS
  + Route53 latency-based routing between ALBs

GCP ADVANTAGE:
• ONE IP address for the entire globe (anycast)
• SSL termination at 175+ edge locations (not just at the region)
• Automatic failover (no Route53 health check delays)
• Built-in Cloud CDN and Cloud Armor at the same layer
```

Part 10 — GCP Cost Management & FinOps

10.1 GCP Pricing Model (vs AWS)

Pricing Feature	GCP	AWS
On-Demand	Per-second billing	Per-second (Linux), per-hour (Windows)
Sustained Use Discounts	**Automatic 30% discount** after using a VM 25%+ of the month	No equivalent — must buy RI/SP

Pricing Feature	GCP	AWS
Committed Use Discounts	1-year (37% off) or 3-year (55% off) resource or spend-based	Reserved Instances (1/3-year), Savings Plans
Spot/Preemptible	Spot VMs (60-91% off, can be preempted anytime)	Spot Instances (similar, 2-min warning)
Free Tier	Always Free (not just 12 months) for many services	12-month free tier + always-free limits
Network Egress	Premium Tier ($0.12/GB) or Standard Tier (0.085/GB)	Single tier ($0.09/GB us-east-1)
BigQuery	Pay-per-query ($5/TB scanned) OR flat-rate slots	Redshift: pay per-cluster-hour (always on)

10.2 GCP Cost Management Tools

Tool	GCP	AWS Equivalent
Cost Visibility	Cloud Billing Reports + BigQuery Export	Cost Explorer + CUR to S3
Budgets & Alerts	Cloud Billing Budgets	AWS Budgets
Recommendations	**Active Assist / Recommender**	Trusted Advisor / Compute Optimizer
Right-Sizing	VM Recommender (auto-suggests right size)	Compute Optimizer
Commitment Management	CUD analysis in Recommender	RI/SP recommendations
Cost Allocation	Labels (≈ AWS Tags) + Billing accounts	Tags + Cost Allocation Tags
FinOps Exports	Billing export to BigQuery (query with SQL!)	CUR export to S3 (query with Athena)

Part 11 — GCP Interview Q&A: 50 Questions

Category 1: GCP Architecture & Design (15 Questions)

Q1: How does GCP's network model fundamentally differ from AWS?

> **A:** GCP uses a **Global VPC** model — a single VPC spans all regions automatically. Subnets are regional, but the VPC itself is global. This means VMs in us-central1 can communicate with VMs in europe-west1 via internal IPs without any peering or Transit Gateway. In contrast, AWS VPCs are regional — you need VPC Peering or Transit Gateway for cross-region communication. This makes GCP's multi-region architectures inherently simpler.

Q2: When would you choose Cloud Spanner over Cloud SQL?

> **A:** Cloud Spanner is the right choice when you need: (1) **Global distribution** with strong consistency — writes in one region are immediately consistent in all regions, (2) **Horizontal scaling** of relational data — Spanner auto-shards while maintaining SQL semantics, (3) **99.999% availability SLA** (five nines), (4) **Transactions across regions** — banking, financial systems, global inventory. Cloud SQL is for standard workloads that don't need global scale (max single-region, limited to 96 vCPUs). **AWS comparison:** There is no true AWS equivalent. Aurora Global Database offers cross-region reads but eventual consistency. DynamoDB Global Tables offer multi-region writes but it's NoSQL. Spanner is uniquely GCP.

Q3: Explain the difference between GKE Standard, GKE Autopilot, and Cloud Run.

> **A:**
>
> - **GKE Standard:** Full Kubernetes control — you manage nodes, choose machine types, configure node pools. Best for complex workloads needing custom operators, service meshes, or specific node configurations. ≈ AWS EKS with managed node groups.
> - **GKE Autopilot:** Google manages everything including nodes. You define pods only, billed per-pod resource request. Best when you need Kubernetes APIs but don't want node operations. ≈ EKS + Fargate but with better auto-scaling and simpler configuration.
> - **Cloud Run:** Serverless containers — no cluster, no nodes, no pods. Scale to zero, pay per request. Best for stateless HTTP services, event-driven workloads, and microservices that don't need K8s complexity. ≈ AWS App Runner but significantly more capable (WebSocket, gRPC, 60-min timeout, traffic splitting).

Q4: What is a Shared VPC and when would you use it?

> **A:** Shared VPC allows an organization to designate a **Host Project** that owns the VPC network, and **Service Projects** that use subnets from that VPC. It centralizes network management (subnets, firewall rules, routes) in one project while allowing workload teams to deploy resources in their own projects. **When to use:** Enterprise environments where a central networking team manages IP allocation, firewall rules, and connectivity while application teams need their own project-level autonomy. **AWS comparison:** The closest equivalent is AWS Resource Access Manager (RAM) sharing subnets across accounts + Transit Gateway, but Shared VPC is simpler since it's one VPC with shared subnets.

Q5: How do you design a multi-region application on GCP?

> **A:** GCP's Global VPC makes this simpler than AWS:
>
> 1. **Global HTTP(S) Load Balancer** with a single anycast IP — automatically routes to the nearest healthy backend
> 2. **GKE clusters or Cloud Run services** in each target region
> 3. **Cloud Spanner** for globally consistent database (or Cloud SQL with cross-region replicas for read scaling)
> 4. **Memorystore** for regional caching
> 5. **Cloud CDN** enabled at the load balancer for static content
> 6. **Cloud DNS** with geo-based routing policies
>
> In AWS, this requires CloudFront + multiple ALBs + Route53 latency routing + Aurora Global Database — more components, more complexity.

Q6: What are VPC Service Controls and why are they critical?

> **A:** VPC Service Controls create a **security perimeter** around GCP projects to prevent data exfiltration. Even if an attacker compromises credentials, they cannot copy data out of the perimeter. The perimeter restricts API-level access — e.g., a BigQuery dataset inside the perimeter cannot be accessed from outside, even with valid IAM permissions. **AWS has no direct equivalent.** The closest is VPC Endpoints + S3 Bucket Policies, but VPC Service Controls are more comprehensive because they work at the API level across all supported services.

Q7: Explain Cloud Interconnect options and when to use each.

> **A:**
>
> - **Dedicated Interconnect:** Direct physical connection to Google's network (10G or 100G). Use when you need >10 Gbps throughput and consistent latency. ≈ AWS Direct Connect.

- **Partner Interconnect:** Connection through a supported service provider (50 Mbps to 50 Gbps). Use when you can't reach a Google colocation facility. ≈ AWS Direct Connect via partner.
- **HA VPN:** Encrypted tunnels over the internet (up to 3 Gbps per tunnel). Use for smaller workloads, backup connectivity, or quick setup. ≈ AWS Site-to-Site VPN.
- **Cross-Cloud Interconnect:** Direct connection between GCP and another cloud (AWS, Azure). ≈ No AWS equivalent (AWS expects you to use their Direct Connect).

Q8: How does BigQuery differ from traditional data warehouses?

A: BigQuery is a **serverless, petabyte-scale** analytics platform that separates storage and compute. Key differences: (1) **No cluster management** — no provisioning, no vacuuming, no index tuning, (2) **Pay-per-query** — $5 per TB scanned, or flat-rate slots for predictable costs, (3) **BigQuery ML** — train ML models using SQL directly in the warehouse, (4) **Real-time streaming** — stream data directly into BigQuery tables, (5) **Federated queries** — query Cloud SQL, Cloud Storage, Bigtable without moving data. **AWS comparison:** Requires Redshift (cluster) + Athena (serverless) + Glue Data Catalog + SageMaker (ML) — four separate services to match what BigQuery does in one.

Q9: What is Anthos and when does it make sense?

A: Anthos is Google's **multi-cloud and hybrid platform** that lets you run GKE-managed Kubernetes clusters on: (1) GCP, (2) On-premises (bare metal/VMware), (3) AWS (Anthos on AWS), (4) Azure (Anthos on Azure). It provides unified management (Config Sync, Policy Controller, Service Mesh) across all environments. **When it makes sense:** Organizations with a true multi-cloud strategy, edge computing needs, or those migrating from on-prem but needing consistent operations. **When it doesn't:** Single-cloud environments or cost-sensitive deployments (Anthos licensing adds cost). **AWS has no equivalent** — AWS Outposts runs AWS on-prem but doesn't run on Azure/GCP.

Q10: How do you implement zero-trust security on GCP?

A: GCP's **BeyondCorp Enterprise** is the native zero-trust solution:

1. **Identity-Aware Proxy (IAP):** Proxies all access to applications. Users must authenticate and pass device/context checks before reaching any application. No VPN needed.
2. **BeyondCorp Enterprise:** Extends IAP with device trust signals, threat protection, and data protection.
3. **VPC Service Controls:** Data perimeters around projects.
4. **Workload Identity Federation:** Service-to-service authentication without service account keys.

5. **Binary Authorization:** Only signed, verified containers can deploy.

AWS comparison: AWS has Verified Access (similar to IAP but newer and less mature), and Systems Manager Session Manager (for SSH replacement). GCP's BeyondCorp is more mature because Google uses it internally for 100K+ employees.

Category 2: GCP Migration & Modernization (10 Questions)

Q11: Walk me through a migration from AWS to GCP.

A: Phase 1: **Assess** — Use StratoZone/mFIT to discover workloads, map AWS services to GCP equivalents, identify dependencies. Phase 2: **Plan** — Deploy GCP Landing Zone (Fabric FAST), establish Cloud Interconnect or HA VPN to AWS, define migration waves. Phase 3: **Deploy** — Wave 1: Non-critical workloads (Migrate to VMs for lift-and-shift). Wave 2: Databases (DMS for homogeneous, manual for heterogeneous like DynamoDB → Firestore). Wave 3: Modernize (EKS → GKE, Lambda → Cloud Functions/Cloud Run). Phase 4: **Optimize** — Right-size with VM Recommender, apply Sustained Use Discounts, move analytics to BigQuery. **Key mapping:** EC2 → Compute Engine, S3 → Cloud Storage, RDS → Cloud SQL, EKS → GKE, Lambda → Cloud Functions/Cloud Run, DynamoDB → Firestore/Bigtable.

Q12: How would you migrate a large Oracle database to GCP?

A: Three options depending on complexity: (1) **Cloud SQL PostgreSQL** — Use DMS with schema conversion for simple schemas. Requires rewriting PL/SQL. (2) **AlloyDB** — PostgreSQL-compatible with 4x better transaction performance and 100x better analytics. Best for Oracle workloads that need both OLTP + OLAP. (3) **Cloud Spanner** — For globally distributed Oracle workloads that need horizontal scaling and strong consistency. Most expensive but most capable. **Tools:** Oracle → PostgreSQL schema conversion with Ora2Pg or DMS built-in. Data migration via DMS continuous replication. **AWS comparison:** AWS equivalent is Aurora PostgreSQL, but AlloyDB's combined OLTP+OLAP capability has no AWS match.

Q13: How do you migrate Kubernetes workloads from EKS to GKE?

A: (1) Export Kubernetes manifests (Deployments, Services, ConfigMaps, Secrets) — these are portable. (2) Translate AWS-specific resources: IRSA → Workload Identity, ALB Ingress → GKE Ingress, EBS CSI → GCE PD CSI, AWS Secrets Manager → GCP Secret Manager. (3) Rebuild CI/CD: CodePipeline → Cloud Build, ECR → Artifact Registry. (4) Migrate data: EBS snapshots → GCE Persistent Disk, S3 → Cloud Storage. (5) Update DNS

> incrementally (canary migration). **Key benefit of moving to GKE:** Auto-upgrade, auto-repair, Autopilot mode, release channels, Binary Authorization — features that require manual configuration on EKS.

Q14: How do you modernize a monolithic application using GCP services?

> **A:** Apply the **Strangler Fig pattern** using GCP services: (1) Deploy the monolith on Compute Engine or GKE as-is. (2) Identify high-value modules to extract first (e.g., authentication, notifications). (3) Extract modules as Cloud Run services or GKE microservices. (4) Use Pub/Sub for async event-driven communication between old and new. (5) Use API Gateway or Cloud Endpoints to route traffic (old endpoints → monolith, new endpoints → microservices). (6) Migrate database incrementally: monolith DB → Cloud SQL, then extract per-service databases. **Key GCP advantage:** Cloud Run makes microservice deployment trivial — push a container, get a URL, automatic scaling.

Q15: What's your approach to migrating a data warehouse from Redshift to BigQuery?

> **A:** (1) **Schema migration:** Export Redshift DDL, convert to BigQuery DDL (handle DISTKEY/SORTKEY removal — BigQuery auto-optimizes). (2) **Data migration:** Export to S3, use Storage Transfer Service to copy to Cloud Storage, then load into BigQuery. For large datasets, use Transfer Appliance. (3) **Query migration:** Convert Redshift SQL to BigQuery SQL (differences: GETDATE() → CURRENT_TIMESTAMP(), TOP N → LIMIT N, materialized views syntax). (4) **ETL migration:** Replace Glue with Dataflow or Dataproc. Replace Athena queries with BigQuery federated queries. (5) **BI migration:** Replace QuickSight with Looker or Connected Sheets. **Key wins:** BigQuery eliminates: cluster sizing, vacuuming, WLM configuration, concurrency scaling setup.

Category 3: GCP DevOps & Operations (10 Questions)

Q16: How would you set up a CI/CD pipeline on GCP?

> **A:** GitHub/GitLab → Cloud Build (triggered on PR/push) → Steps: unit tests → SAST → Docker build → push to Artifact Registry → Binary Authorization attestation → deploy to GKE (via Config Sync GitOps) or Cloud Run (direct deploy). For infrastructure: GitHub → Cloud Build → Terraform plan/apply → deploy GCP resources. **Key difference from AWS:** Cloud Build combines CodeBuild + CodePipeline in one service. Binary Authorization is native (no need for separate signing tool). Config Sync provides native GitOps without installing ArgoCD.

Q17: How do you implement GitOps on GKE?

A: Use **Config Sync** (part of Anthos Config Management): (1) Store K8s manifests in a Git repo. (2) Install Config Sync on GKE cluster. (3) Config Sync continuously reconciles cluster state with Git. (4) Any manual `kubectl` changes are automatically reverted. (5) Policy Controller (OPA Gatekeeper) enforces policies before sync. **AWS comparison:** On EKS, you install ArgoCD (third-party). Config Sync is Google-managed, auto-upgraded, and integrates with GCP IAM. Same GitOps principles, less operational overhead.

Q18: How do you monitor and alert for a production GKE application?

A: (1) **Cloud Monitoring:** GKE auto-exports pod/node metrics. Create dashboards for CPU, memory, request latency, error rate. (2) **Cloud Logging:** Structured logging from containers auto-collected. Create log-based metrics for custom alerts. (3) **Cloud Trace:** Distributed tracing with OpenTelemetry SDK integration. (4) **Error Reporting:** Auto-groups exceptions from Cloud Logging. (5) **SLO Monitoring:** Define SLIs (latency P99 < 200ms, error rate < 0.1%) and create SLO-based alerts that fire on error budget burn rate. **The SLO-based alerting is unique to GCP** — AWS CloudWatch doesn't have built-in SLO burn rate alerting.

Q19: How do you handle secrets management on GCP?

A: Secret Manager for storing secrets (API keys, passwords, certificates). Access via: (1) GKE: Mount as volumes using Secret Manager CSI driver, or use Workload Identity to access API. (2) Cloud Run: Mount as environment variables or volumes (native integration). (3) Cloud Functions: Access via client library + Workload Identity. **Key difference from AWS:** Secret Manager integrates natively with Cloud Run (zero-config), and Workload Identity eliminates service account key files entirely. **AWS comparison:** Secrets Manager + EKS needs External Secrets Operator or AWS Secrets CSI driver.

Q20: How do you implement disaster recovery on GCP?

A: DR strategy depends on RTO/RPO:

- **Backup & Restore (RTO: hours, RPO: hours):** Cloud SQL automated backups (cross-region), Cloud Storage dual-region/multi-region buckets, GKE backup for clusters.
- **Pilot Light (RTO: minutes, RPO: minutes):** Minimal infrastructure in DR region, Cloud SQL cross-region read replicas promoted on failover, pre-built Terraform for scaling up.
- **Warm Standby (RTO: seconds, RPO: seconds):** Active infrastructure in both regions, Global HTTP(S) Load Balancer auto-routes on health check failure, Cloud Spanner for zero-RPO database.

- **Active-Active (RTO: 0, RPO: 0):** Global Load Balancer serving from multiple regions, Cloud Spanner for globally consistent writes, Memorystore in each region.

GCP advantage: Global Load Balancer + Cloud Spanner makes active-active significantly simpler than AWS (where you need Route53 + Aurora Global DB + custom conflict resolution).

Category 4: GCP Security & Compliance (8 Questions)

Q21: How do you enforce compliance across a GCP organization?

A: Layered approach: (1) **Organization Policies** (preventive): Restrict regions, disable external IPs, require Shielded VMs, block service account key creation. (2) **VPC Service Controls** (data exfiltration prevention): Create perimeters around sensitive projects. (3) **Security Command Center Premium** (detective): Continuous vulnerability scanning, compliance monitoring (CIS, PCI, ISO 27001), threat detection. (4) **Policy Controller** on GKE: OPA Gatekeeper policies for pod security. (5) **Binary Authorization**: Only signed containers deploy. (6) **Cloud Audit Logs**: Always-on, tamper-evident audit trail. **AWS comparison:** SCPs + Config Rules + Security Hub + GuardDuty — similar layered approach but VPC Service Controls has no AWS equivalent.

Q22: What is the difference between Organization Policies and IAM?

A: Organization Policies define **what CAN be done** (guardrails on resources). IAM defines **who CAN do it** (access control on identities). Example: Organization Policy says "VMs cannot have external IPs" — this applies regardless of IAM permissions. Even an org admin cannot create a VM with an external IP if the policy is set. IAM says "user@company.com has roles/compute.admin" — they can manage VMs but are still constrained by Organization Policies. **AWS equivalent:** SCPs (Organization Policies) vs IAM Policies.

Q23: How do you secure data in BigQuery?

A: (1) **Column-level security:** Tag sensitive columns with policy tags, grant access per tag. (2) **Row-level security:** Filter rows based on user identity using row-level access policies. (3) **Dynamic data masking:** Mask PII fields for non-privileged users. (4) **CMEK:** Customer-managed encryption keys via Cloud KMS. (5) **VPC Service Controls:** Prevent BigQuery data from being exported outside the perimeter. (6) **Audit logging:** All queries logged in Cloud Audit Logs. (7) **Authorized views:** Share specific views without exposing underlying tables. **AWS comparison:** Redshift has column-level grants and dynamic data

masking, but no equivalent to VPC Service Controls for preventing data exfiltration via query results.

Q24: How does Workload Identity Federation work?

> **A:** Workload Identity Federation allows external identities (GitHub Actions, AWS, Azure AD, OIDC providers) to impersonate GCP service accounts **without creating service account keys**. Flow: (1) External workload authenticates to its identity provider (e.g., GitHub OIDC). (2) Presents its token to GCP's Security Token Service (STS). (3) STS validates the token against the configured Workload Identity Pool. (4) Returns a short-lived GCP access token bound to a service account. **Key benefit:** Zero static credentials. The token is short-lived, bound to the specific workload, and audited. **AWS equivalent:** OIDC Identity Provider + `AssumeRoleWithWebIdentity`. Same concept, GCP implementation is slightly more flexible (supports AWS as an identity provider too).

Q25: How do you implement encryption on GCP?

> **A:** Three layers: (1) **Default encryption:** All data encrypted at rest by default with Google-managed keys (AES-256). No configuration needed. (2) **CMEK (Customer-Managed Encryption Keys):** Customer controls keys in Cloud KMS. Supports key rotation, access controls, and audit logging. Required for regulatory compliance. (3) **CSEK (Customer-Supplied Encryption Keys):** Customer provides the encryption key directly — Google never stores it. Used for maximum control but customer must manage key availability. (4) **Confidential Computing:** Encrypts data **in-use** (not just at-rest and in-transit). Uses AMD SEV or Intel TDX. **AWS comparison:** Default encryption (both), CMK/KMS (both), CloudHSM (both). Confidential Computing is more mature on GCP (Confidential VMs, Confidential GKE nodes, Confidential Dataflow).

Category 5: GCP Cost & FinOps (7 Questions)

Q26: How does GCP's pricing differ from AWS?

> **A:** Key differences: (1) **Sustained Use Discounts:** GCP automatically gives up to 30% off VMs used 25%+ of the month. AWS requires explicit RI/SP purchases. (2) **Committed Use Discounts:** 1-year (37%) or 3-year (55%) commitments. Can be spend-based (flexible) or resource-based (specific). More flexible than AWS RIs. (3) **Per-second billing:** Both GCP and AWS bill per-second for Linux. (4) **Network pricing:** GCP offers Premium Tier (Google's private backbone, $0.12/GB) vs Standard Tier (public internet, 0.085/GB). AWS has one tier. (5) **BigQuery:** Pay-per-query ($5/TB) is unique — AWS Redshift charges per-cluster-hour regardless of usage. (6) **Free Tier:** GCP has an "Always Free" tier (e.g., 1

f1-micro VM, 5 GB Cloud Storage) that never expires. AWS free tier mostly expires after 12 months.

Q27: How do you optimize costs on GCP?

A: (1) **Active Assist / Recommender:** Auto-suggests VM right-sizing, idle resource deletion, CUD purchases. (2) **Sustained Use Discounts:** Already automatic. (3) **CUDs:** Commit to 1 or 3 years for predictable workloads. (4) **Spot VMs:** Use for fault-tolerant batch workloads (60-91% savings). (5) **GKE Autopilot:** Pay per-pod, avoid over-provisioned nodes. (6) **Cloud Run:** Scale-to-zero for intermittent workloads. (7) **BigQuery:** Use partitioned/clustered tables to reduce scan costs. (8) **Storage lifecycle:** Auto-transition to Nearline/Coldline/Archive. (9) **Billing export to BigQuery:** Analyze spend with SQL queries for deeper insights. (10) **Committed Use Discount analysis:** Use Recommender to identify optimal commitment levels.

Q28: How would you set up FinOps reporting on GCP?

A: (1) Export billing data to BigQuery (native integration, free). (2) Create a **Looker dashboard** or **Connected Sheets** for executive reporting. (3) Tag resources with labels (team, environment, cost-center, project). (4) Set up **Cloud Billing Budgets** with alerts at 50%, 80%, 100%. (5) Enable **Recommender** for right-sizing and CUD recommendations. (6) Create a BigQuery SQL report:

```
SELECT
  project.id,
  service.description,
  SUM(cost) AS total_cost,
  SUM(IFNULL(credits.amount, 0)) AS total_credits
FROM `billing_export.gcp_billing_export_v1_XXXXX`
GROUP BY 1, 2
ORDER BY total_cost DESC
```

AWS comparison: AWS CUR export → S3 → Athena → QuickSight. GCP's BigQuery integration is simpler (no Glue crawler needed, SQL directly on billing data).

Category 6: Scenario-Based & Behavioral (10 Questions)

Q29: A startup wants to build a real-time analytics platform. AWS or GCP? Why?

A: GCP for this use case. Reason: BigQuery + Pub/Sub + Dataflow is a significantly simpler stack. The startup gets: (1) Pub/Sub for event ingestion (replaces Kinesis + SQS +

SNS), (2) Dataflow for stream processing (replaces Kinesis Data Analytics or custom Spark), (3) BigQuery for analytics and ML (replaces Redshift + Athena + SageMaker), (4) Looker for visualization (replaces QuickSight). Total: 4 services vs 7+ on AWS. For a startup with limited DevOps capacity, fewer services = faster time-to-market and lower operational burden.

Q30: A bank needs a globally distributed transaction system. Which GCP service and why?

A: Cloud Spanner. It's the only relational database that offers: (1) Global distribution with strong consistency (external consistency, the strongest guarantee). (2) Horizontally scalable writes across regions. (3) 99.999% availability SLA (five nines). (4) Full SQL support with transactions across regions. A bank processing transactions across US, EU, and APAC can have writes in any region be immediately consistent everywhere. **AWS comparison:** Aurora Global Database offers cross-region reads but asynchronous replication (eventual consistency for writes). DynamoDB Global Tables offers multi-region writes but with eventual consistency and NoSQL limitations. Neither matches Spanner's globally consistent SQL transactions.

Q31: Your GKE cluster is experiencing intermittent pod failures. How do you troubleshoot?

A: (1) `kubectl get events --sort-by='.lastTimestamp'` — Check for OOMKilled, ImagePullBackOff, CrashLoopBackOff. (2) Cloud Logging: Filter by resource.type="k8s_container" and severity="ERROR". (3) Cloud Monitoring: Check node CPU/memory, pod resource requests vs limits, HPA metrics. (4) If OOMKilled: Increase memory limits or fix memory leak. (5) If CrashLoopBackOff: Check container logs, verify health/readiness probes, check config maps and secrets. (6) If scheduling failures: Check node pool capacity, Karpenter/cluster autoscaler configuration. (7) **GKE-specific:** Check if node auto-repair triggered (GKE auto-repairs unhealthy nodes). Check GKE release channel for automatic upgrades that might have caused issues. (8) Cloud Trace: Look for latency spikes indicating downstream service failures.

Q32: How would you design a multi-cloud architecture spanning GCP and AWS?

A: (1) **Networking:** Cross-Cloud Interconnect (GCP-specific service) for dedicated connectivity between GCP and AWS, or HA VPN as backup. (2) **Compute:** Anthos on GKE (GCP) and Anthos on AWS (run GKE in AWS VPC) for consistent K8s management. (3) **Identity:** Workload Identity Federation — GKE on AWS authenticates to GCP services without static keys. (4) **Data:** Primary database on one cloud (source of truth), read replicas or CDC (Change Data Capture) to the other cloud. (5) **DNS:** Split DNS — Cloud DNS for GCP resources, Route53 for AWS resources, conditional forwarding between them. (6)

Monitoring: Unified with Datadog or Grafana Cloud (neither cloud's native monitoring covers both well). (7) **IaC:** Terraform with separate providers but shared module patterns.

Q33: The CTO asks: "Should we go all-in on GCP or stay multi-cloud?" What's your advice?

A: It depends on the organizational context:

- **Go all-in on GCP if:** Primary workloads are data/analytics/ML (BigQuery + Vertex AI advantage), Kubernetes-first strategy (GKE is best), minimal legacy AWS investment, team is willing to learn GCP deeply.
- **Stay multi-cloud if:** Regulatory requirement for multi-cloud DR, existing heavy AWS investment that would be costly to migrate, M&A bringing in different cloud workloads, vendor lock-in avoidance is a board-level concern.
- **Hybrid approach (recommended for most):** Primary workloads on the cloud that best fits (GCP for data, AWS for breadth), secondary cloud for DR and specific workloads, Anthos for unified Kubernetes management, Terraform for infrastructure portability.

Architect's advice: "Don't be multi-cloud for the sake of it. Every additional cloud doubles operational complexity, training costs, and security surface area. Be multi-cloud only when the business requires it."

Q34: A healthcare company needs HIPAA compliance on GCP. How do you architect it?

A: (1) **BAA:** Sign a Business Associate Agreement (BAA) with Google Cloud. (2) **Organization Policies:** Restrict resource locations to approved regions, require CMEK encryption, disable external IPs. (3) **VPC Service Controls:** Create a perimeter around all projects handling PHI — prevent data exfiltration even with valid credentials. (4) **Encryption:** CMEK for all data stores (Cloud SQL, Cloud Storage, BigQuery). Enable CMEK for Cloud Logging. (5) **Access:** VPN or Cloud Interconnect for access (no public endpoints). Identity-Aware Proxy for admin access. (6) **Audit:** Cloud Audit Logs exported to a separate locked-down logging project. Retained for 7+ years per HIPAA requirement. (7) **Network:** Private Google Access, no external IPs, VPC firewall rules restricting all ingress. (8) **GKE:** Shielded GKE nodes, Binary Authorization, Workload Identity (no service account keys). **AWS comparison:** Similar controls — BAA, HIPAA-eligible services, KMS, CloudTrail, VPC. GCP's VPC Service Controls add an extra layer AWS doesn't have.

Q35: How do you evaluate the total cost of running a workload on GCP vs AWS?

A: Use a structured comparison: (1) **Compute:** Map instance types (e.g., m6i.xlarge → e2-standard-4). Factor in GCP Sustained Use Discounts (automatic 30% off). (2) **Storage:** Compare S3 vs Cloud Storage pricing by tier. (3) **Database:** RDS/Aurora vs Cloud

SQL/AlloyDB. Include license costs for Oracle/SQL Server. (4) **Networking:** AWS egress ($0.09/GB)vsGCPPremium$(0.12/GB) vs GCP Standard ($0.085/GB$).$Considerintra-regionandcross-regiontransfercosts.(5) **Managedservices:** **CompareBigQuery(pay-per-query)vsRedshift(always-oncluster).(6) **Commitments:** **Compare1-yearRIvs1-yearCUDsavings.(7) **UseGCPPricingCalculator**and**AWSPricingCalculator**forside-by-side.(8) **Don'tforgethiddencosts:** **AWSNATGateway$(0.045/GB processed), GCP Cloud NAT ($0.045/GB), VPN/Interconnect costs. **Pro tip:** Export both cloud bills to a single dashboard (Looker/Grafana) for ongoing comparison.

Q36-Q50: Rapid-Fire Q&A

#	Question	Answer (GCP)	AWS Comparison
36	Max VMs in a single project?	Quota-based (~24 vCPUs default, can increase)	Similar (account-level quotas)
37	How to do blue-green deployments?	Cloud Run traffic splitting, GKE + Istio	CodeDeploy / ArgoCD rollouts
38	How to handle DDoS?	Cloud Armor (WAF + DDoS)	Shield + WAF (two services)
39	How to manage DNS?	Cloud DNS (public/private zones)	Route 53
40	How to do IaC on GCP?	**Terraform** (de facto standard)	Terraform / CloudFormation
41	How to manage container images?	Artifact Registry	ECR
42	How to do serverless ETL?	Dataflow (Apache Beam)	Glue
43	How to queue messages?	Pub/Sub (replaces SQS+SNS+EventBridge)	SQS + SNS + EventBridge
44	How to run Apache Spark?	Dataproc (managed Spark/Hadoop)	EMR
45	How to handle file storage (NFS)?	Filestore	EFS
46	How to do API management?	Apigee (enterprise) / Cloud Endpoints	API Gateway
47	How to handle batch jobs?	Cloud Run Jobs / Batch	AWS Batch

#	Question	Answer (GCP)	AWS Comparison
48	How to manage ML experiments?	Vertex AI Experiments	SageMaker Experiments
49	How to handle IoT?	**Partner solutions** (GCP deprecated IoT Core)	IoT Core + Greengrass
50	How to handle video transcoding?	Transcoder API	Elastic Transcoder / MediaConvert

Quick Reference — GCP Certification Path

Certification	Level	Focus
Cloud Digital Leader	Foundational	Cloud concepts, GCP overview
Associate Cloud Engineer	Associate	Deploy and manage infrastructure
Professional Cloud Architect	Professional	Design and plan cloud solutions
Professional Cloud DevOps Engineer	Professional	CI/CD, SRE, monitoring
Professional Cloud Security Engineer	Professional	Security, compliance, identity
Professional Data Engineer	Professional	BigQuery, Dataflow, ML pipelines
Professional Machine Learning Engineer	Professional	Vertex AI, ML design patterns
Professional Cloud Network Engineer	Professional	VPC, Load Balancing, Interconnect
Professional Cloud Database Engineer	Professional	Cloud SQL, Spanner, AlloyDB, Bigtable